An Adaptive Logarithm Quantization Method for DNN Compression

Yuan Wang[1], Zhaoliang He[2], Chen Tang[3], Zhi Wang[3]✉, and Wenwu Zhu[2]✉

[1] Tsinghua-Berkeley Shenzhen Institute, Tsinghua University, China
wangyuan19@mails.tsinghua.edu.cn
[2] Department of Computer Science and Technology, Tsinghua University, China
hezl19@mails.tsinghua.edu.cn, wwzhu@tsinghua.edu.cn
[3] Shenzhen International Graduate School, Tsinghua University, China
tc20@mails.tsinghua.edu.cn, wangzhi@sz.tsinghua.edu.cn

Abstract. The size and complexity of Neural Network models grow rapidly in recent years, which makes the inference of these models require more computational and memory resources. To reduce the required resources, quantization is one of the promising methods. Logarithm quantization can both reduce the model size and the computational complexity because the time-consuming multiplication operation can be replaced with the addition operation in logarithm domain. However, the previous logarithm quantization methods use a fixed logarithm base. Therefore, they cannot adapt according to the distribution of data and bit-width budgets, which causes performance degradation. To address such a problem, we propose an adaptive quantization method to optimize the quantization function. Our method first finds an optimized weight quantization function by minimizing the quantization loss of the model's weight data under a given bit-width budget. Then we use a zero-shot way to find an optimized quantization function for activation data. Finding the optimized parameters is time-consuming. We propose a heuristic algorithm to solve the optimization problem fast. Compared to the previous logarithm quantization methods, our method can achieve up to 72.53% higher Top-1 accuracy under the same bit-width constraint.

Keywords: Neural Network · Logarithm Quantization · Optimization · Zero-shot Model Compression.

1 Introduction

Nowadays, we witness a prosperous development of Neural Network (NN), especially deep Neural Network (DNN) technology. The DNN models show great performance in varieties of tasks, such as image classification and object detection. To obtain better inference performance, the number of layers of DNN models becomes larger and the structures become more complex [6]. Training and deploying these large and complex DNN models require a huge amount of resources [4]. However, in reality, we sometimes need to deploy the models in a cheaper and resource-limited edge device, such as a mobile device [14]. Thus, the

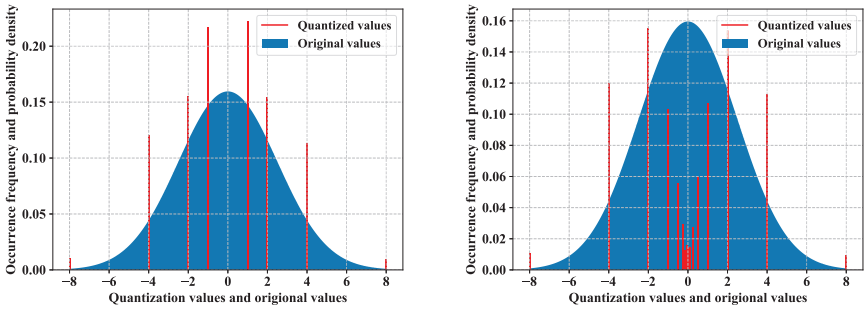
deployment of these models arises the problem of how to compress the existed models to satisfy the resource constraints on the edge devices while preserving the good performance of the original DNN models and keeping the inference procedure within an acceptable time limit. The main resources that influence the inference time are memory bandwidth and the amount of available computational resources [13].

To address such a problem, one promising method is to quantize the weight or activation data of DNN models to lower bit-width representation. After quantization, both the memory and the computational resource requirement will be reduced. However, the quantization procedure may result in the DNN models' performance degradation compared to the original ones [3].

To address this, many quantization technologies are proposed. The quantization methods can be classified into two categories based on whether retraining is required [10] or not. One is quantization-aware training method. The other one is post-training quantization. Another classification bases on the quantization function. The quantization function can be linear or non-linear. Among the non-linear quantization functions, the logarithm quantization function is a promising one because the time-consuming multiplication operation can be replaced with a more efficient addition operation in the logarithm domain [9].

However, the traditional logarithm quantization method uses a fixed base quantization function, which cannot adapt to different models and bit-width constraints. As shown in Fig. 1, the original float point data (the blue part, representing the density) are quantized to several quantization values (the red part, representing the occurrence frequency) by a quantization function.

Comparing Fig. 1(a) and Fig. 1(b), the quantization values remain the same for the larger data when the quantization bit width changes.



(a) Quantization result using 3 bits. (b) Quantization result using 4 bits.

Fig. 1. Quantization results of a fixed base quantization function under different bit-width constraints.

Therefore, we propose a quantization scheme to address the above problems. Our method adds a new degree of freedom by automatically and adaptively

selecting FSR and the base of the logarithm quantization function to preserve the performance of the DNN models better. In particular, our contributions are as follows.

- The logarithm quantization is analyzed. How the different factors influence the logarithm quantization results is shown.
- We propose an adaptive logarithm quantization scheme that can adjust the quantization function according to the distribution of DNN models' parameters, the distribution of activation data and bit-width constraints.
- We design a fast heuristic searching algorithm to find the parameters of the quantization function.
- A more efficient quantization method for activation data is designed.

2 Methodology

2.1 Loss Function

In most previous post-training quantization works, the loss function is designed as follows: $J = \|w - Q(w)\|_2^2$. The w is the original data, and the $Q(\cdot)$ is the quantization function that quantizes the w to a lower bit width representation w_q. The w can be the weight data or activation data in a DNN model. We need to find the best quantization function $Q(\cdot)$ to reduce the quantization loss.

2.2 Logarithm Quantization

For the basic logarithm quantization method, the quantization function is as follows:

$$\mathrm{LogQuant}(x, \mathrm{bitwidth}, \mathrm{FSR}) = s \cdot 2^{\tilde{x}} \tag{1}$$

$$\tilde{x} = \mathrm{Clip}(\mathrm{Round}(\log_2 x), \mathrm{FSR} - 2^{\mathrm{bitwidth}-1}, \mathrm{FSR}) \tag{2}$$

$$\mathrm{Clip}(x, \min, \max) = \begin{cases} \min + 1 & x \leq \min \\ \max & x \geq \max \\ x & \text{otherwise} \end{cases} \tag{3}$$

where bitwidth means how many bits are used to represent the quantized data, FSR stands for full scale range, and s stands for the sign of x.

As shown in Fig. 2, when we change the base of logarithm function to a smaller one $\sqrt{2}$ and compare with Fig. 1(b), the quantization results are more suitable for the bit-width constraint.

To find the best quantization function for different models and bit widths, we proposed a quantization algorithm that adaptively chooses the base of the logarithm quantization function to minimize the quantization loss. The quantization function is revised as:

$$\mathrm{LogQuant}(x, \mathrm{bitwidth}, \mathrm{FSR}, b) = s \cdot b^{\tilde{x}} \tag{4}$$

$$\tilde{x} = \text{Clip}(\text{Round}(\log_b x), \text{FSR} - 2^{\text{bitwidth}-1}, \text{FSR}) \tag{5}$$

Such revision influences the representation range of the quantization function. When the bit width remains the same and the maximum quantization value $m = b^{FSR}$ remains the same value by adjusting FSR, the quantization range R can be calculated as $R = m(1 - \frac{1}{b^{2^{\text{bitwidth}-1}}})$. We can figure out that a smaller base provides denser quantization results, but the quantization range is narrow. Both the density and the quantization range influence the quantization error.

2.3 Analysis using Random Data

The weight data of a DNN model follow nearly normal distribution with zero mean [6]. For models that use Batch Normalization (BN) layers, the BN layers make the activation data nearly normally distributed with zero means [8]. That helps us make use of a set of standard normal distribution and change the scaling factor to simulate the different data distributions However, such scaling operation introduces a scaling factor σ to the original data.

Assuming a set of original data ϕ follows a zero-mean normal distribution as $\phi \sim \mathcal{N}(0, \sigma^2)$, we can transform ϕ to a standard normal distribution as follows: $\frac{\phi}{\sigma} \sim \mathcal{N}(0, 1)$. The converse transform can be done as follows: $\sigma\varphi \sim \mathcal{N}(0, \sigma^2)$, where data φ follows standard normal distribution as $\varphi \sim \mathcal{N}(0, 1)$.

Given a set of random data, the loss function is not derivable in many points. As shown in Fig. 3, the quantization loss for one data has several sudden changes when the base changes. For different quantization data values, the sudden changes occur at different quantization base points. In addition, the number of the weights data is large. That means the sudden change will appear almost everywhere and the derivation of loss function will frequently disappear when the base changes. That also makes it very time-consuming to apply a grid search approach.

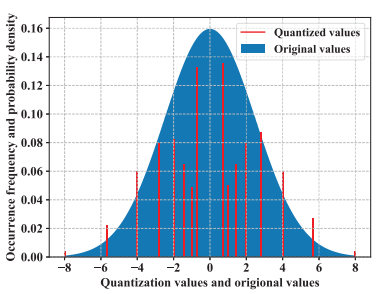


Fig. 2. Quantization results using 4 bits and a fixed logarithm function with a lower base.

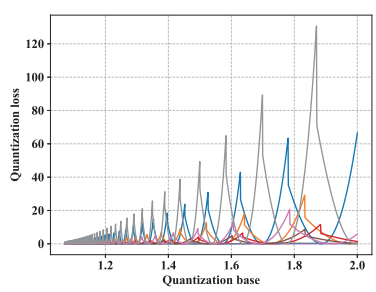


Fig. 3. Quantization loss for eight random values when quantization base changes.

To test the influence of the scaling factor, we simulate how the quantization loss changes along with the scaling factor changing. The results are shown in Fig. 4.

We can see that the quantization errors seesaw around a mean value while the scaling factor changes. However, compared to the influence of the base of the quantization function, the scaling factor makes an ignorable contribution to the quantization procedure. The scaling step helps us find the FSR for the quantization function.

To better understand the relationship between the quantization loss and the quantization base, we apply the change of base formula: $\log_b x = \frac{\log_2 x}{\log_2 b} = c \cdot \log_2 x, c = \frac{1}{\log_2 b}$. We call c the revised quantization base. The quantization function can be rewritten as:

$$\text{LogQuant}(x, \text{bitwidth}, c, M) = s \cdot M \cdot 2^{\frac{\tilde{x}}{c}} \tag{6}$$

$$\tilde{x} = \text{Clip}(\text{Round}(c \cdot \log_2 \frac{x}{M}), -2^{\text{bitwidth}-1}, 0) \tag{7}$$

Then the relativity between quantization loss and revised quantization base c is shown in Fig. 5.

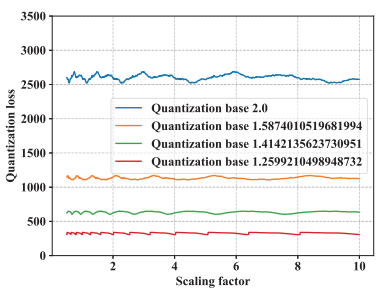


Fig. 4. Quantization loss for the data set $\varphi \sim \mathcal{N}(0, 1)$ with different scaling factors and quantization bases.

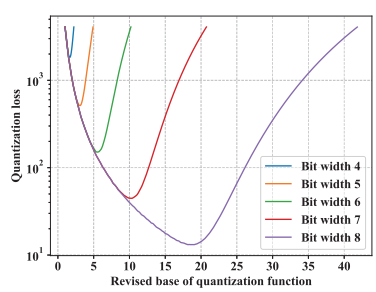


Fig. 5. Quantization loss for the data set $\varphi \sim \mathcal{N}(0, 1)$ when the revised quantization base changes.

The minimal quantization loss is the global minimum of the function.

According to the results above, we assume that the quantization loss is a nearly "cup" like function of the parameter c. Therefore, we propose a heuristic algorithm to find the best quantization base for the quantization function.

Step 1: scaling the data a using the maximum absolute value M of data. Choose a stop threshold th.

Step 2: random choose an initial revised base c for the quantization function (6) and an initial step length l.

Step 3: set the base of quantization function to $c-l$, c, and $c+l$. Then we get three different quantization function $Q_1(\cdot)$, $Q_2(\cdot)$ and $Q_3(\cdot)$. Quantize the data using these three quantization functions and compute the quantization losses.

Step 4: Compare three losses and find the quantization function of the lowest and the second lowest quantization loss. According to different results, select a new initial revised base. Repeat step3 and step4 until the end condition.

2.4 Quantization for Activation Data

We adopt the method from reference [2] to generate a batch of representative data x^r. Then the data x^r is input into the original model. For activation data of each layer, we get a batch of representative results. We can therefore apply our optimization algorithm to the results to find the optimized quantization function.

3 Evaluation

3.1 Effectiveness of the Searching Method

We use Intel E5-2650 CPU to run our algorithm and record the time that different searching methods use. The result shows that the searching time of our algorithm is significantly shorter than the grid search method.

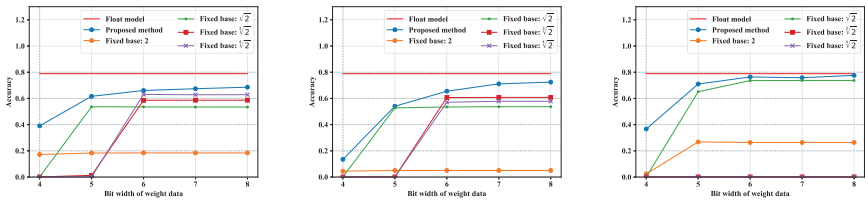
Table 1. Time consumption comparison between the proposed algorithm and grid search method on two generated data sets that follow standard normal distribution.

Dataset size	Method	Grid search number									
		100	200	300	400	500	600	700	800	900	1000
10000	Proposed	0.103	0.105	0.103	0.104	0.226	0.233	0.102	0.104	0.101	0.232
	Grid search	1.20	2.34	3.42	4.66	5.71	6.96	8.02	9.34	10.4	11.7
10000000	Proposed	7.85	7.87	7.85	8.10	7.86	7.85	7.87	23.4	24.5	23.5
	Grid search	113	226	339	457	568	679	799	996	1190	1336

3.2 Accuracy Results

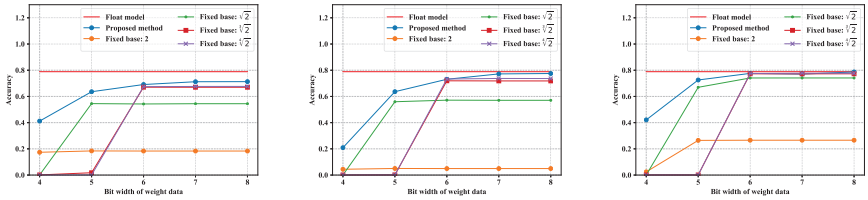
We select three CNN models (ResNet18/50 [7], InceptionV3 [11]) to test our method due to that CNN is one kind of the most common DNN model. These models are all pretrained on the ImageNet data set. We apply our quantization method to both weight data and activation data.

The results of different bit-width representations of activation data and weight data for ResNet18/50 and InceptionV3 are shown in Fig. 6 and Fig. 7. We compare the accuracy results of our quantization method with those of the basic quantization method using a fixed base quantization function as decreased in [9] combining with the method in [2].



(a) ResNet18 (b) ResNet50 (c) InceptionV3

Fig. 6. Top-1 accuracy of different models under different quantization bit-width conditions for weight data when the activation data are quantized using 4 bits.



(a) ResNet18 (b) ResNet50 (c) InceptionV3

Fig. 7. Top-1 accuracy of different models under different quantization bit-width conditions for weight data when the activation data are quantized using 8 bits.

From the results, we can see that our method provides a better accuracy result in most of the cases. Especially when the bit width goes up, our method can benefit a lot compared to the fixed base method. When the quantization bit widths for weight data and activation data are both 8, our method shows almost no performance degradation. The fixed base method is only suitable for some cases of bit-width constraints and models. However, our method can find a better quantization function adaptively according to different bit-width constraints and models.

4 Conclusion

In this paper, we propose a new post-training logarithm quantization method. We design an algorithm to find the best quantization parameters faster rather than use the cumbersome grid search method. We test the effectiveness of our searching method and conclude that our searching method can significantly reduce the searching time. Then we test our quantization method by both quantizing the weight data and activation data. The results show that our method can adaptively find a better logarithm quantization function for different bit-width constraints and models. Our method can quantize the CNN models with lower quantization loss and less performance degradation.

Acknowledgements. This work is supported in part by NSFC (Grant No. 61872215), and Shenzhen Science and Technology Program (Grant No. RCYX-20200714114523079).

References

1. Cai, J., Takemoto, M., Nakajo, H.: A deep look into logarithmic quantization of model parameters in neural networks. In: Proceedings of the 10th International Conference on Advances in Information Technology. IAIT 2018, Association for Computing Machinery, New York, NY, USA (2018).
2. Cai, Y., Yao, Z., Dong, Z., Gholami, A., Mahoney, M.W., Keutzer, K.: ZeroQ: A novel zero shot quantization framework. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (June 2020)
3. Chen, Y., Zhang, K., Gong, C., Hao, C., Zhang, X., Li, T., Chen, D.: T-DLA: An open-source deep learning accelerator for ternarized DNN models on embedded FPGA. In: 2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). pp. 13–18 (2019).
4. Deng, L., Li, G., Han, S., Shi, L., Xie, Y.: Model compression and hardware acceleration for neural networks: A comprehensive survey. Proceedings of the IEEE **108**(4), 485–532 (2020).
5. Gong, C., Chen, Y., Lu, Y., Li, T., Hao, C., Chen, D.: VecQ: Minimal loss DNN model compression with vectorized weight quantization. IEEE Transactions on Computers pp. 1–1 (2020).
6. Gong, C., Li, T., Lu, Y., Hao, C., Zhang, X., Chen, D., Chen, Y.: μL2Q: An ultra-low loss quantization method for DNN compression. In: 2019 International Joint Conference on Neural Networks (IJCNN). pp. 1–8 (2019).
7. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (June 2016)
8. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. ArXiv **abs/1502.03167** (2015)
9. Miyashita, D., Lee, E.G., Murmann, B.: Convolutional neural networks using logarithmic data representation. ArXiv **abs/1603.01025** (2016)
10. Nogami, W., Ikegami, T., O'uchi, S., Takano, R., Kudoh, T.: Optimizing weight value quantization for CNN inference. In: 2019 International Joint Conference on Neural Networks (IJCNN). pp. 1–8 (2019).
11. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (June 2016)
12. Vogel, S., Liang, M., Guntoro, A., Stechele, W., Ascheid, G.: Efficient hardware acceleration of CNNs using logarithmic data representation with arbitrary log-base. In: Proceedings of the International Conference on Computer-Aided Design. ICCAD '18, Association for Computing Machinery, New York, NY, USA (2018).
13. Wang, E., Davis, J.J., Zhao, R., Ng, H.C., Niu, X., Luk, W., Cheung, P.Y.K., Constantinides, G.A.: Deep neural network approximation for custom hardware: Where we've been, where we're going. ACM Comput. Surv. **52**(2) (May 2019).
14. Wang, K., Liu, Z., Lin, Y., Lin, J., Han, S.: HAQ: Hardware-aware automated quantization with mixed precision. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 8604–8612 (2019).